技术部第一次技术分享会

技术部 2026 年活动总览

姚雨森　技术部分管主席

2026-10-16

在技术部的一年，能收获什么

这一年，技术部采取**技术分享会**与**小组协作开发**相结合的形式。

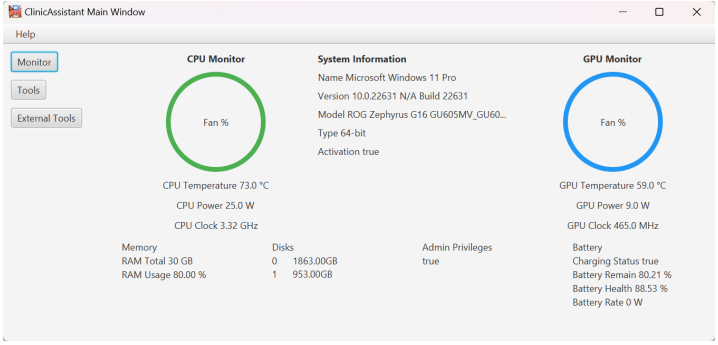
- 学会几门编程语言，掌握常用的开发工具。
- 在真实项目中动手开发，获得实践经验。
- 对技术有自己的理解和品味。
- 认识新的同学，和大家一起交流、共同进步。

这一年，技术部做什么

- **第一学期：软件开发的基本认识和实践**
  学习软件开发基础，通过重写一个简单项目积累实践经验。
- **寒假：开拓视野**
  学习其他编程语言和开发工具，了解软件开发的各个方向。
- **第二学期：真实项目实战**
  按照自身兴趣，参与协会真实生产项目的开发。

ClinicAssistant：电脑诊所工具箱

- 查看电脑的软硬件信息和运行状态。
- 启动一些常用工具。



先看读数：每个数值都有一个编号

ClinicAssistant 借助 LibreHardwareMonitor（简称 LHM）读取硬件信息。LHM 用索引和数值的表来储存各个硬件信息。

索引（编号）	当前数值（示例）
7	50
8	25
20	12

哪个指标是 CPU 温度?

再查名称：CPU 温度对应哪个编号

LHM 辅助程序还提供一份清单，列出**指标名称与编号的对应关系**。

指标名称	索引（编号）
CPU 温度	7
CPU 功率	8
已用内存	20

这个表说明 CPU 温度对应的索引是 7。接下来我们使用索引 7 向 LHM 查询，即可得到需要的数值。

根据索引读取对应的数值

清单告诉我们指标的名称和索引；要取得**当前数值**，还需要按索引请求读取。

1. **查到 CPU 温度对应的索引是 7**
   名称清单中，CPU 温度对应的编号是 7。
2. **把索引 7 发给 LHM 辅助程序**
   ClinicAssistant 请求：“请读取索引 7 对应的当前数值。”
3. **收到返回值 50**
   请求的是 CPU 温度，因此这次读到的温度是 50 °C。

用编号读取数值，再保存到程序里

ClinicAssistant 更新读数，再按记住的编号取值。



- 程序知道自己问的是 CPU 温度，就把 50 存到对应位置。
- 下次读到 51，就把保存的温度更新成 51。

同一个流程，可以有不同的实现

刚才展示了一种做法。仅仅是**什么时候查找索引**，就有多种选择。

1. **每次读取前都查找**
   每次获取 CPU 温度，都先从清单中找到编号，再读取数值。
2. **第一次用到时查找并保存**
   后续直接使用已存的编号；需要判断这个指标是否已经查过。
3. **程序启动时集中查找**
   先找齐所有需要的编号，后续直接使用；启动时要做更多准备。

重复查找的开销、准备工作的时机、代码的复杂程度，都需要权衡。

第一学期：在小任务中练习设计

第一学期主要就是围绕这样的流程，练习设计、实现和改进。

- **重写一个已有功能**
  每次选取流程中的一小部分，小组讨论如何设计、如何实现。
- **阅读并理解已有代码**
  理解代码如何组织、如何设计、如何实现，也可以试着找找原有的 bug 和不足。
- **整个过程都在小组内分工进行**
  从阅读代码、讨论设计，到编写、检查和整合，都由组员分工协作。

第一学期怎么安排

1. **前两次分享会：认识基本概念，做简单练习**
   安装开发工具，熟悉基本操作。
2. **第二次分享会后：分组，开始项目**
   每组五人，由一位部长指导；结合空闲时间和组队意愿安排。
3. **后两次分享会及之后：边学边做**
   把 ClinicAssistant 的功能拆成小任务，逐步重新实现。
   每周统一给出任务，组会上回顾进展、讨论问题。

组会每周不超过 1 小时，独立写代码每周不超过 3 小时。

分享会之外，怎么学

分享会：建立整体认识

- 有哪些知识，它们怎样关联。
- 一个工具为什么存在，能解决什么问题。

课后自学与实践：掌握具体内容

- 根据学习资料，自主学习具体知识和用法。
- 通过练习和小任务，把学到的内容用起来。

我们会提供入门资料，由部长指导小组实践。

遇到问题，可以在群里提问，也可以带到组会上一起讨论。

程序的功能，是用代码写出来的

无论是读取 CPU 温度，还是更新界面上的文字，背后都有代码。

源代码 ⟶ **编译、解释等处理** ⟶ **计算机执行**

- 代码描述要做什么，以及怎样做。
- 经过编译器、解释器等工具处理，计算机才能执行这些要求。
- 软件还会用到图片、配置等资源，代码把它们组织起来。

编程语言，就是我们向计算机表达要求的语言。

编程语言有很多种

不同语言有不同的设计特点，也有各自常见的使用场景。

- **C：**接近底层，常用于系统和嵌入式开发。
- **Java：**支持跨平台，常用于服务端和桌面应用。
- **Python：**语法简洁，常用于自动化和数据分析。

这些场景并不是语言的使用边界，同一个问题也可以用不同语言解决。

技术部第一学期学习 Java，并用它实践 ClinicAssistant 的开发。

开发一个程序，需要掌握什么

可以把学习软件开发，类比为学习用一门语言写文章。

- **编程语言：知道代码怎么写**
  就像学习词汇和语法，先能写出正确的句子。
- **开发工具：会编辑、运行和调试程序**
  就像写作时会使用字典、文字编辑工具。
- **组织代码：会拆分功能、安排结构**
  就像学习谋篇布局和修辞，让表达清楚、便于理解。

只会语法，不代表能写好文章；只会语言，也不代表能写好程序。

开发工具：IDEA 与 Git

写程序时，既要编辑和运行代码，也要管理不断发生的修改。

IDE：集成开发环境

我们使用 IntelliJ IDEA。

- **编写：**补全代码、提示错误。
- **运行：**启动程序、查看结果。
- **调试：**暂停程序、检查数值。

就像写作时使用文字编辑软件。

Git：版本管理工具

记录代码怎样一步步改过来。

- **记录：**保存选定的修改。
- **追溯：**查看差异、找回旧代码。
- **协作：**合并修改、处理冲突。

就像保留修订记录、合并多人稿件。

例如：用 IDEA 排查温度显示异常，再用 Git 记录这次修复。

同样的功能，可以有不同的代码组织

以 CPU、GPU、硬盘三个位置显示温度为例，两种写法都能实现。

现在修改需求：把 `50 °C` 改为 `50.0 °C`，统一保留一位小数。

写法一：各处重复实现

- 三处各写一遍小数和单位处理。
- **阅读：**逐处查看，确认处理规则是否一致。
- **修改：**三处分别改，漏一处就会显示不一致。

写法二：提取共用函数

- 写一个“格式化温度”函数，三个位置都调用它。
- **阅读：**名字说明用途，处理细节集中查看。
- **修改：**改一处，三处一起更新。

写代码也像写文章：除了表达正确，还要组织清楚，方便修改。